

LAB 3: XÂY DỰNG TÍNH NĂNG QUẢN LÝ THÔNG TIN NGƯỜI DÙNG

1. Mục tiêu bài Lab

- Biết cách nhìn vào một yêu cầu giao diện (UI) và phân rã nó thành các Component nhỏ.
- Áp dụng thành thạo mô hình **Smart Component (Container)** và **Dumb Component (Presentational)**.
- Quản lý state cục bộ và luồng dữ liệu (Data Flow) một chiều thông qua props mà không cần gọi API thật.

2. Mô tả Tình huống (Scenario)

Bạn cần xây dựng một trang "Hồ sơ của tôi" cho khách hàng. Giao diện yêu cầu hai trạng thái:

1. **Chế độ xem (View Mode):** Hiển thị Avatar, Họ tên, Email, và Số điện thoại. Có một nút "Chỉnh sửa".
2. **Chế độ sửa (Edit Mode):** Hiển thị form nhập liệu để người dùng thay đổi Họ tên và Số điện thoại. Có nút "Lưu" và "Hủy bỏ".

Cấu trúc thư mục dự án React (Khuyến nghị)

Trong các dự án React hiện đại (khởi tạo bằng Vite hoặc Create React App), toàn bộ code của bạn sẽ nằm trong thư mục src. Dưới đây là cách tổ chức các thư mục con:

```
my-react-app/
├── node_modules/      # (Tự động sinh ra) Chứa các thư viện
├── public/             # Chứa file index.html, favicon, hình ảnh tĩnh
├── src/                # THƯ MỤC CODE CHÍNH
│   ├── assets/        # Hình ảnh, CSS, fonts dùng chung
│   ├── components/    # Chứa các component dùng chung (Atoms, Molecules)
│   │   ├── atoms/     # Nút bấm (Button), Ô nhập liệu (Input)...
│   │   └── molecules/ # SearchBar, Menu...
│   ├── features/      # Chia theo từng CỤM TÍNH NĂNG (Gộp Smart & Dumb)
│   │   └── UserProfile/# TÍNH NĂNG HỒ SƠ NGƯỜI DÙNG (Bài Lab)
│   │       ├── ProfileView.jsx      (Dumb)
│   │       ├── ProfileForm.jsx      (Dumb)
│   │       └── UserProfileManager.jsx (Smart)
│   ├── hooks/         # Chứa các Custom Hooks (useUserData...)
│   └── pages/          # Chứa các trang (Home, About, Dashboard)
```

LAB 3: XÂY DỰNG TÍNH NĂNG QUẢN LÝ THÔNG TIN NGƯỜI DÙNG

		App.jsx	# Component gốc bọc toàn bộ ứng dụng
		main.jsx	# File entry point (Vite) hoặc index.js (CRA)
		package.json	# Chứa cấu hình và danh sách thư viện
		vite.config.js	# Cấu hình Vite (nếu dùng Vite)

Tại sao lại có thư mục features? Thay vì nhét hàng trăm component vào thư mục components, việc gom nhóm theo tính năng (Feature-based) giúp dự án gọn gàng hơn. Toàn bộ code liên quan đến "Hồ sơ người dùng" (cả Smart và Dumb) sẽ nằm chung một chỗ.

Hướng dẫn cài đặt và chạy thử bài Lab

Để sinh viên có thể xem trực tiếp kết quả bài Lab trên trình duyệt, hãy thực hiện theo các bước sau. Chúng ta sẽ sử dụng **Vite** (công cụ build React nhanh và phổ biến nhất hiện nay).

Bước 1: Khởi tạo dự án

Mở Terminal (hoặc Command Prompt) và chạy lệnh sau để tạo dự án mới:

```
Bash
```

```
npm create vite@latest my-react-app -- --template react
```

```
cd my-react-app
```

```
npm install
```

Bước 2: Phân tích và tạo component (Xem bên dưới)

Tạo các file và copy đoạn code từ bài Lab trước vào đúng vị trí thư mục như thiết kế:

1. Tạo thư mục: src/features/UserProfile/
2. Tạo file ProfileView.jsx và dán code Dumb component số 1 vào.
3. Tạo file ProfileForm.jsx và dán code Dumb component số 2 vào.
4. Tạo file UserProfileManager.jsx và dán code Smart component vào.

Bước 3: Gọi Component vào App.jsx

Mở file src/App.jsx, xóa các code mặc định đi và gọi Smart Component (UserProfileManager) ra để hiển thị.

```
import React from 'react';
import UserProfileManager from '../features/UserProfile/UserProfileManager';

function App() {
  return (
    <div style={{ padding: '20px', backgroundColor: '#f9f9f9', minHeight: '100vh' }}>
      <h1 style={{ textAlign: 'center' }}>Hệ thống quản lý cửa hàng</h1>
    </div>
  );
}
```

```
<hr />

{/* Khởi chạy tính năng User Profile */}
<UserProfileManager />

</div>
);
}
```

```
export default App;
```

Bước 4: Khởi động server ảo

Quay lại Terminal và chạy lệnh:

```
npm run dev
```

Terminal sẽ hiển thị một đường link (thường là <http://localhost:5173>). Hãy nhấn Ctrl + Click vào đường link đó để mở trình duyệt.

PHẦN 1: PHÂN TÍCH VÀ CHIA NHỎ COMPONENT

Dựa theo nguyên tắc Đơn trách nhiệm (Single Responsibility) và Smart/Dumb, chúng ta chia giao diện này thành 3 component chính:

Tên Component	Phân loại	Vai trò & Trách nhiệm
UserProfileManager	Smart	Chứa "dữ liệu giả" (Mock Data). Quản lý state xem đang ở chế độ View hay Edit. Định nghĩa hàm lưu dữ liệu.
ProfileView	Dumb	Chỉ nhận dữ liệu người dùng và hàm onEdit qua props để hiển thị thông tin và nút bấm.
ProfileForm	Dumb	Nhận dữ liệu ban đầu để đưa vào form. Có state cục bộ chỉ để quản lý nội dung đang nhập. Gọi hàm onSave hoặc onCancel qua props.

PHẦN 2: TRIỂN KHAI CODE

Xây dựng Dumb Component - ProfileView

Component này chỉ làm một việc duy nhất là in dữ liệu ra màn hình.

```
import React from 'react';
```

```
const ProfileView = ({ user, onEdit }) => {
  return (
    <div style={{ border: '1px solid #ddd', padding: '20px', borderRadius: '8px' }}>
      <h3>Thông tin cá nhân</h3>
      <div style={{ display: 'flex', alignItems: 'center', marginBottom: '15px' }}>
        <img
          src={user.avatar}
          alt="Avatar"
          style={{ width: '80px', height: '80px', borderRadius: '50%', marginRight:
'15px' }}
        />
        <div>
          <p><strong>Họ và tên:</strong> {user.name}</p>
          <p><strong>Email:</strong> {user.email}</p>
          <p><strong>Số điện thoại:</strong> {user.phone}</p>
        </div>
      </div>
      <button
        onClick={onEdit}
        style={{ padding: '8px 16px', cursor: 'pointer', backgroundColor: '#007bff',
color: '#fff', border: 'none', borderRadius: '4px' }}
      >
        Chỉnh sửa thông tin
      </button>
    </div>
  );
};

export default ProfileView;
```

Xây dựng Dumb Component - ProfileForm

Component này có chứa useState, nhưng đây là **UI State** (state tạm thời của form nhập liệu), không phải state gốc của ứng dụng.

```
import React, { useState } from 'react';

const ProfileForm = ({ user, onSave, onCancel }) => {
  // Tạo state tạm để lưu trữ dữ liệu người dùng đang gõ trên form
  const [formData, setFormData] = useState({
    name: user.name,
    phone: user.phone,
  });
};
```

```

const handleChange = (e) => {
  setFormData({
    ...formData,
    [e.target.name]: e.target.value
  });
};

const handleSubmit = (e) => {
  e.preventDefault();
  onSave(formData); // Đẩy dữ liệu mới lên Smart Component xử lý
};

return (
  <div style={{ border: '1px solid #ddd', padding: '20px', borderRadius: '8px' }}>
    <h3>Chỉnh sửa thông tin</h3>
    <form onSubmit={handleSubmit}>
      <div style={{ marginBottom: '10px' }}>
        <label style={{ display: 'block', marginBottom: '5px' }}>Họ và tên:</label>
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
          style={{ width: '100%', padding: '8px' }}
        />
      </div>
      <div style={{ marginBottom: '15px' }}>
        <label style={{ display: 'block', marginBottom: '5px' }}>Số điện
thoại:</label>
        <input
          type="text"
          name="phone"
          value={formData.phone}
          onChange={handleChange}
          style={{ width: '100%', padding: '8px' }}
        />
      </div>

      <button type="submit" style={{ padding: '8px 16px', marginRight: '10px',
backgroundColor: '#28a745', color: '#fff', border: 'none', borderRadius: '4px' }}>
        Lưu thay đổi
      </button>
      <button type="button" onClick={onCancel} style={{ padding: '8px 16px',
backgroundColor: '#6c757d', color: '#fff', border: 'none', borderRadius: '4px' }}>

```

```
        Hủy bỏ
      </button>
    </form>
  </div>
);
};

export default ProfileForm;
```

Xây dựng Smart Component - UserProfileManager

Đây là bộ não điều phối. Nó giữ State thực sự của dữ liệu người dùng (mô phỏng database) và quyết định việc hiển thị form hay hiển thị thông tin.

```
import React, { useState } from 'react';
import ProfileView from './ProfileView';
import ProfileForm from './ProfileForm';

const UserProfileManager = () => {
  // 1. Quản lý Mock Data (Thay thế cho việc fetch API)
  const [userData, setUserData] = useState({
    id: 'U001',
    name: 'Nguyễn Văn A',
    email: 'nva@example.com',
    phone: '0901234567',
    avatar: 'https://via.placeholder.com/80'
  });

  // 2. Quản lý UI State: Quyết định xem đang ở chế độ xem hay sửa
  const [isEditing, setIsEditing] = useState(false);

  // 3. Logic xử lý lưu dữ liệu
  const handleSaveProfile = (updatedData) => {
    console.log("Đang gọi API cập nhật dữ liệu...", updatedData);

    // Cập nhật state gốc với dữ liệu mới
    setUserData(prevData => ({
      ...prevData,
      name: updatedData.name,
      phone: updatedData.phone
    }));

    // Đóng form
    setIsEditing(false);
    alert('Cập nhật thông tin thành công!');
  };
};
```

```
};

return (
  <div style={{ maxWidth: '500px', margin: '40px auto', fontFamily: 'sans-serif' }}>
    <h2>Quản lý tài khoản</h2>

    {/* 4. Điều phối hiển thị (Conditional Rendering) */}
    {isEditing ? (
      <ProfileForm
        user={userData}
        onSave={handleSaveProfile}
        onCancel={() => setIsEditing(false)}
      />
    ) : (
      <ProfileView
        user={userData}
        onEdit={() => setIsEditing(true)}
      />
    )}
  </div>
);
};

export default UserProfileManager;
```

Các kịch bản kiểm thử (Test Cases) trên giao diện

Sau khi giao diện đã chạy lên, bạn hãy hướng dẫn học viên tự kiểm tra các luồng dữ liệu (Data Flow) bằng cách thao tác trên màn hình:

Kịch bản 1: Kiểm tra chế độ xem (View Mode)

- **Hành động:** Vừa mở trang web lên.
- **Kỳ vọng:** Nhìn thấy Component ProfileView hiển thị. Thấy thông tin mặc định (Nguyễn Văn A, avatar, số điện thoại) và một nút "Chỉnh sửa thông tin". Form nhập liệu **không** xuất hiện.

Kịch bản 2: Kích hoạt chế độ sửa (Edit Mode)

- **Hành động:** Bấm vào nút "Chỉnh sửa thông tin".
- **Kỳ vọng:** Component ProfileView biến mất, Component ProfileForm xuất hiện thay thế. Các ô input đã có sẵn dữ liệu cũ (Nguyễn Văn A, 0901234567). Nút "Lưu thay đổi" và "Hủy bỏ" hiển thị.

Kịch bản 3: Hủy bỏ chỉnh sửa

- **Hành động:** Sửa tên thành "Nguyễn Văn B", sau đó bấm nút "Hủy bỏ".
- **Kỳ vọng:** Quay lại giao diện View Mode ban đầu. Tên hiển thị vẫn là "Nguyễn Văn A" (vì chưa bấm lưu, state gốc ở Smart Component chưa bị thay đổi).

Kịch bản 4: Lưu dữ liệu mới thành công

- **Hành động:** Bấm lại "Chỉnh sửa", sửa tên thành "Trần Thị B", sửa số điện thoại thành "0999888777". Bấm nút "Lưu thay đổi".
- **Kỳ vọng:** Xuất hiện thông báo alert "Cập nhật thông tin thành công". Tự động quay lại giao diện View Mode, nhưng lúc này thông tin hiển thị đã được cập nhật thành tên và số điện thoại mới.

TỔNG KẾT BÀI LAB

Qua bài lab này, bạn có thể thấy rõ:

- ProfileView và ProfileForm hoàn toàn không biết cấu trúc dữ liệu tổng thể hay cách lưu vào database. Chúng rất "ngốc" và chỉ phản ứng lại tương tác của người dùng (nhấn nút, gõ phím), sau đó báo cáo lên cấp trên qua props.
- UserProfileManager đóng vai trò nhạc trưởng, không chứa thẻ HTML phức tạp nào (ngoài vài thẻ div bọc ngoài), mà tập trung vào luồng dữ liệu (Data Flow) userData và isEditing.